In [8]:
```
from textblob import TextBlob
import nltk
nltk.download('punkt')
nltk.download('stopwords') # For removing common words
nltk.download('averaged_perceptron_tagger') # For Part-of-Speech tagging (used i
nltk.download('wordnet') # For lemmatization
! python -m textblob.download_corpora
# Sample reviews with associated sentiment (as provided)
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\thoma\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\thoma\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     C:\Users\thoma\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\thoma\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
Finished.
[nltk_data] Downloading package brown to
[nltk_data]     C:\Users\thoma\AppData\Roaming\nltk_data...
[nltk_data]   Package brown is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\thoma\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\thoma\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     C:\Users\thoma\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package conll2000 to
[nltk_data]     C:\Users\thoma\AppData\Roaming\nltk_data...
[nltk_data]   Package conll2000 is already up-to-date!
[nltk_data] Downloading package movie_reviews to
[nltk_data]     C:\Users\thoma\AppData\Roaming\nltk_data...
[nltk_data]   Package movie_reviews is already up-to-date!
```

In [9]:
```
# Importation des bibliothèques
import pandas as pd
import numpy as np
import re
import string
from collections import Counter

import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['figure.dpi'] = 120

# Téléchargement des ressources NLTK
import nltk
nltk.download('punkt',                          quiet=True)
nltk.download('punkt_tab',                      quiet=True)
```

```
nltk.download('stopwords',                    quiet=True)
nltk.download('averaged_perceptron_tagger',   quiet=True)
nltk.download('averaged_perceptron_tagger_eng', quiet=True)
nltk.download('wordnet',                      quiet=True)
nltk.download('brown',                        quiet=True)

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from nltk.corpus import wordnet
from nltk import pos_tag

from textblob import TextBlob
from wordcloud import WordCloud

# Chargement des données
avis_clients = pd.read_csv('../datasets/amazon-review.csv')
print(f'Données chargées : {avis_clients.shape[0]:,} lignes, {avis_clients.shape
avis_clients.head()
```

```
Données chargées : 20,000 lignes, 2 colonnes
```

Out[9]:

	reviewText	Positive
0	This is a one of the best apps acording to a b...	1
1	This is a pretty good version of the game for ...	1
2	this is a really cool game. there are a bunch ...	1
3	This is a silly game and can be frustrating, b...	1
4	This is a terrific game on any pad. Hrs of fun...	1

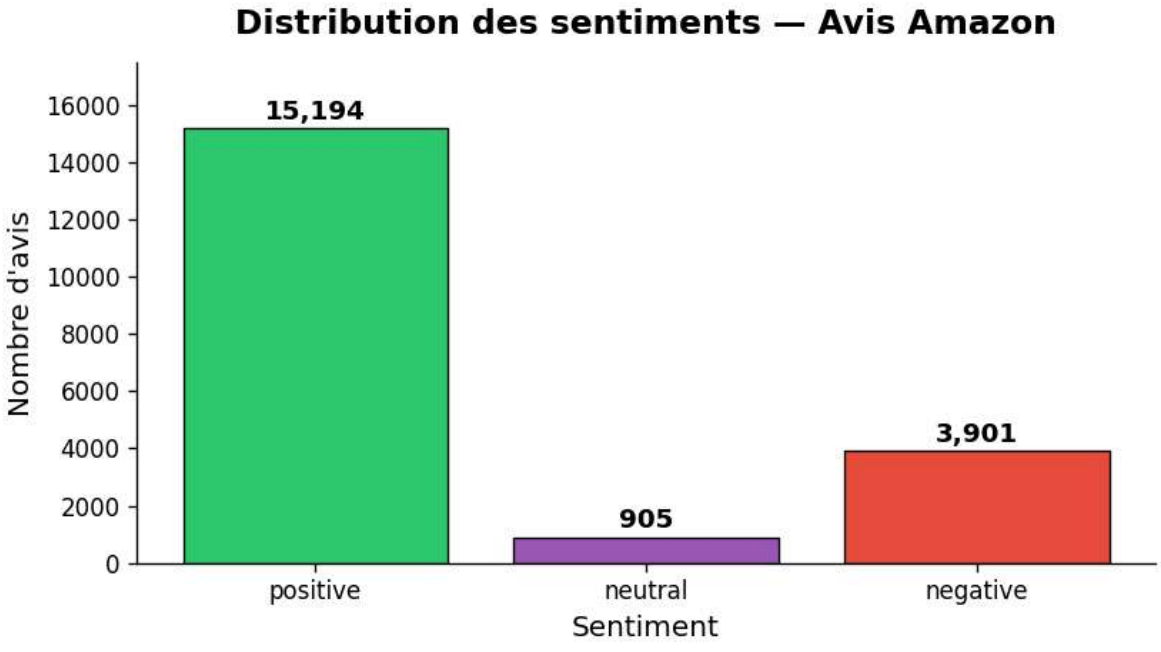
1. Can you perform a sentiment analysis on the collection of product reviews, classify each one as 'positive', 'negative', or 'neutral', and then generate a bar chart to visualize the distribution of these sentiments?"
2. convert the text to lowercase, remove punctuation and numbers, filter out common stopwords, and then lemmatize the remaining words using their appropriate part-of-speech tags.
3. fter the text has been fully preprocessed and lemmatized, could you calculate the frequency of each unique word and then display a bar graph showing the top 15 most common words?
4. generate a word cloud based on the entire corpus of lemmatized review words to provide a visual summary of the most prominent terms.
5. Visualize the frequency of nouns, verbs, adjectives in this text.

1 - Classer chaque avis comme positif, négatif ou neutre et afficher la distribution.

In [10]:
```
# Fonction de classification du sentiment
def classer_sentiment(texte):
    polarite = TextBlob(str(texte)).sentiment.polarity
```

```
    if polarite > 0:
        return 'positive'
    elif polarite < 0:
        return 'negative'
    else:
        return 'neutral'

# Application sur tous les avis
avis_clients['Sentiment'] = avis_clients['reviewText'].apply(classer_sentiment)

# Comptage par catégorie
repartition_sentiment = avis_clients['Sentiment'].value_counts().reindex(['posit
print(repartition_sentiment)

# Graphique à barres
couleurs_sentiment = ['#2ecc71', '#9b59b6', '#e74c3c']

fig, ax = plt.subplots(figsize=(7, 4))
barres = ax.bar(repartition_sentiment.index, repartition_sentiment.values,
                color=couleurs_sentiment, edgecolor='black', linewidth=0.8)

# Affichage des valeurs au-dessus des barres
for barre, val in zip(barres, repartition_sentiment.values):
    ax.text(barre.get_x() + barre.get_width() / 2, barre.get_height() + 100,
            f'{val:,}', ha='center', va='bottom', fontsize=11, fontweight='bold'

ax.set_title('Distribution des sentiments — Avis Amazon', fontsize=14, fontweigh
ax.set_xlabel('Sentiment', fontsize=12)
ax.set_ylabel("Nombre d'avis", fontsize=12)
ax.set_ylim(0, repartition_sentiment.max() * 1.15)
ax.spines[['top', 'right']].set_visible(False)
plt.tight_layout()
plt.show()
```

```
Sentiment
positive    15194
neutral       905
negative     3901
Name: count, dtype: int64
```



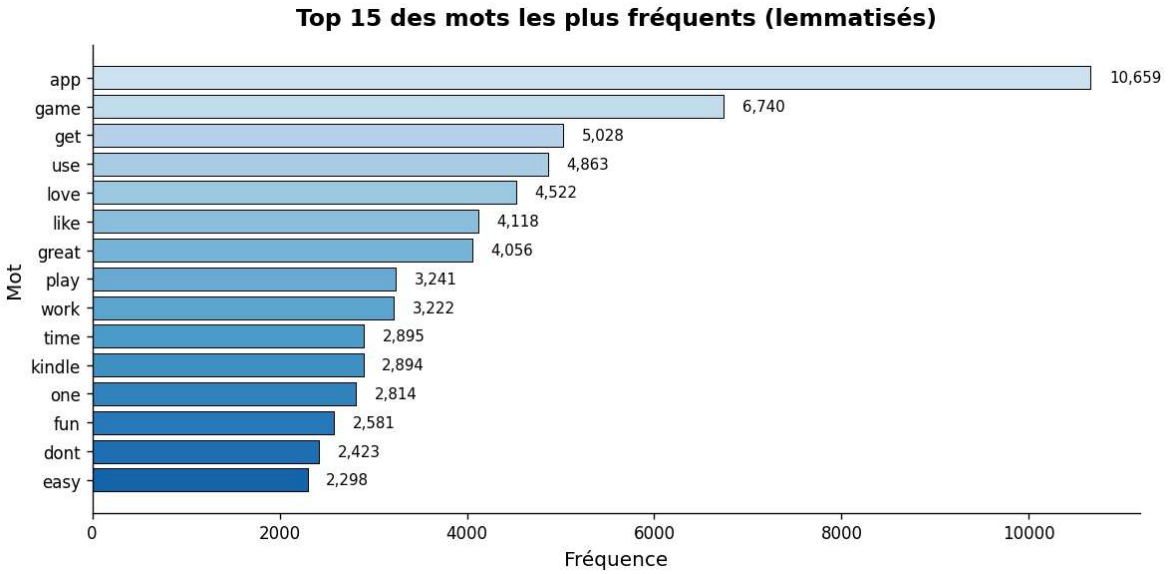
2 - Convertir le texte en minuscule, supprimer la ponctuation et les chiffres, filtrer les motes vides courantset lemmatisez les mots restant en utilisant leurs étiquettes grammaticales.

In [11]:
```
# Initialisation du lemmatiseur et des mots vides
lemmatiseur = WordNetLemmatizer()
mots_vides  = set(stopwords.words('english'))

# Conversion des tags POS Penn Treebank vers WordNet
def convertir_tag_pos(tag_treebank):
    if tag_treebank.startswith('J'):
        return wordnet.ADJ
    elif tag_treebank.startswith('V'):
        return wordnet.VERB
    elif tag_treebank.startswith('N'):
        return wordnet.NOUN
    elif tag_treebank.startswith('R'):
        return wordnet.ADV
    else:
        return wordnet.NOUN

# Pipeline complet de prétraitement
def nettoyer_et_lemmatiser(texte):
    # Minuscules
    texte = str(texte).lower()
    # Suppression des chiffres
    texte = re.sub(r'\d+', '', texte)
    # Suppression de la ponctuation
    texte = texte.translate(str.maketrans('', '', string.punctuation))
    # Tokenisation
    tokens = word_tokenize(texte)
    # Filtrage des mots vides et tokens courts
    tokens = [t for t in tokens if t not in mots_vides and len(t) > 2]
    # Étiquetage POS
    tags_pos = pos_tag(tokens)
    # Lemmatisation avec le bon tag POS
    lemmes = [lemmatiseur.lemmatize(mot, convertir_tag_pos(tag)) for mot, tag in
    return lemmes

print('Prétraitement en cours... (peut prendre 1-2 minutes)')
avis_clients['lemmes'] = avis_clients['reviewText'].apply(nettoyer_et_lemmatiser

print('\nExemple (premier avis) :')
print('Original :', avis_clients['reviewText'].iloc[0][:120])
print('Lemmes   :', avis_clients['lemmes'].iloc[0][:20])
```

```
Prétraitement en cours... (peut prendre 1-2 minutes)

Exemple (premier avis) :
Original : This is a one of the best apps acording to a bunch of people and I agr
ee it has bombs eggs pigs TNT king pigs and realus
Lemmes   : ['one', 'best', 'apps', 'acording', 'bunch', 'people', 'agree', 'bom
b', 'egg', 'pig', 'tnt', 'king', 'pig', 'realustic', 'stuff']
```

3 - Calculer la fréquence de chaque mot unique et afficher un graphique à barres montrant les 15 mots les plus fréquents.

In [12]:
```
# Aplatissement de tous les lemmes en une liste unique
tous_les_lemmes = [mot for liste in avis_clients['lemmes'] for mot in liste]

# Comptage des fréquences
frequence_mots = Counter(tous_les_lemmes)
top_15 = frequence_mots.most_common(15)
mots_top, comptes_top = zip(*top_15)

# Graphique à barres horizontal
fig, ax = plt.subplots(figsize=(10, 5))
palette_bleu = plt.cm.Blues_r(np.linspace(0.2, 0.8, 15))
barres_top = ax.barh(mots_top[::-1], comptes_top[::-1], color=palette_bleu, edge

# Affichage des valeurs
for barre, val in zip(barres_top, comptes_top[::-1]):
    ax.text(barre.get_width() + 200, barre.get_y() + barre.get_height() / 2,
            f'{val:,}', va='center', fontsize=9)

ax.set_title('Top 15 des mots les plus fréquents (lemmatisés)', fontsize=14, fon
ax.set_xlabel('Fréquence', fontsize=12)
ax.set_ylabel('Mot', fontsize=12)
ax.spines[['top', 'right']].set_visible(False)
plt.tight_layout()
plt.show()

print('\nTop 15 :')
for mot, compte in top_15:
    print(f'  {mot:<20} {compte:>7,}')
```

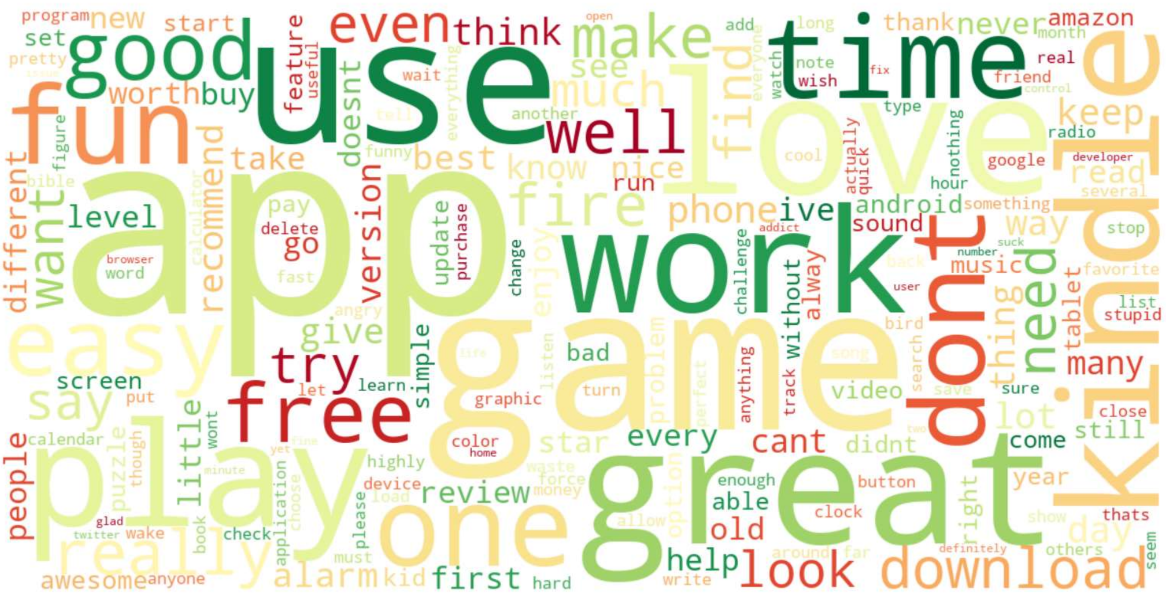


```
Top 15 :
  app                    10,659
  game                    6,740
  get                     5,028
  use                     4,863
  love                    4,522
  like                    4,118
  great                   4,056
  play                    3,241
  work                    3,222
  time                    2,895
  kindle                  2,894
  one                     2,814
  fun                     2,581
  dont                    2,423
  easy                    2,298
```

4 - Génération d'un nuage de mots basé sur l'ensemble du fichier afin de fournir un résumé visuel des termes les plus importants.

In [13]:
```python
# Création du corpus sous forme de chaîne de caractères
corpus_lemmatise = ' '.join(tous_les_lemmes)

# Génération du nuage de mots
nuage_mots = WordCloud(
    width=1200,
    height=600,
    background_color='white',
    colormap='RdYlGn',
    max_words=200,
    collocations=False,
).generate(corpus_lemmatise)

# Affichage
fig, ax = plt.subplots(figsize=(14, 7))
ax.imshow(nuage_mots, interpolation='bilinear')
ax.axis('off')
ax.set_title('Nuage de mots — Avis Amazon lemmatisés', fontsize=16, fontweight='
plt.tight_layout()
plt.show()
```

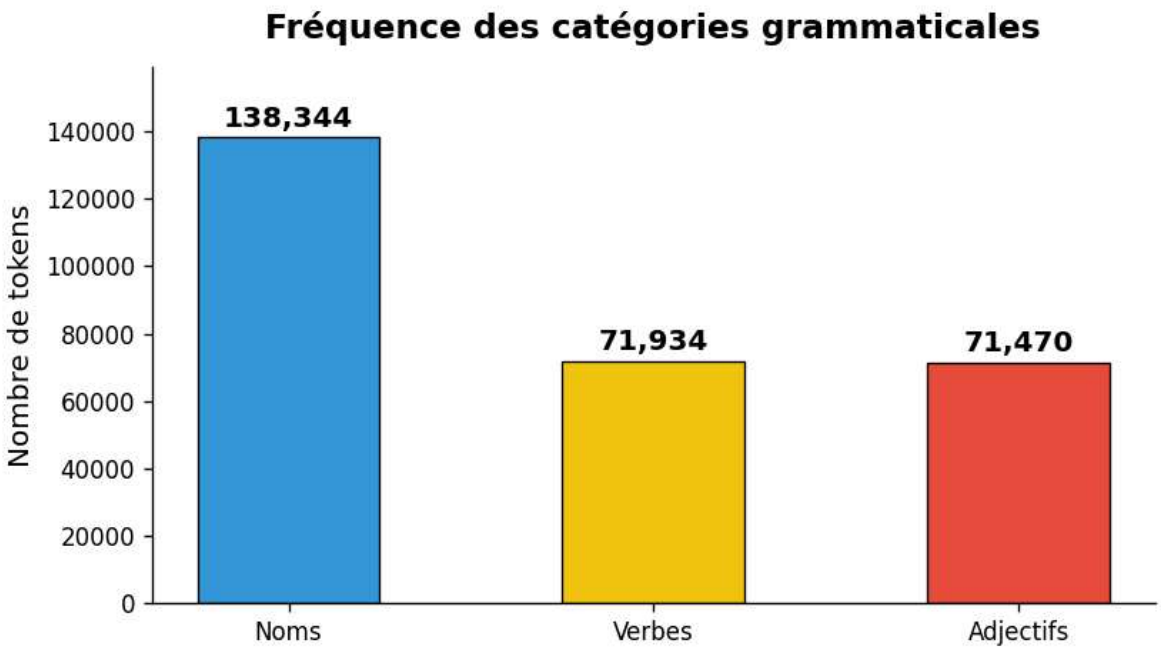
Nuage de mots — Avis Amazon lemmatisés



5 - Visualiser la fréquence des noms, des verbes et des adjectifs dans le fichier.

In [14]:
```
# Fonction de tokenisation simple (sans lemmatisation)
def tokeniser_texte(texte):
    texte = str(texte).lower()
    texte = re.sub(r'\d+', '', texte)
    texte = texte.translate(str.maketrans('', '', string.punctuation))
    tokens = word_tokenize(texte)
    return [t for t in tokens if t not in mots_vides and len(t) > 2]

print('Étiquetage POS en cours...')

# Aplatissement de tous les tokens
tous_tokens = [mot for tokens in avis_clients['reviewText'].apply(tokeniser_text

# Étiquetage POS sur l'ensemble du corpus
etiquettes_pos = pos_tag(tous_tokens)

# Comptage par catégorie grammaticale
nb_noms      = sum(1 for _, tag in etiquettes_pos if tag.startswith('NN'))
nb_verbes    = sum(1 for _, tag in etiquettes_pos if tag.startswith('VB'))
nb_adjectifs = sum(1 for _, tag in etiquettes_pos if tag.startswith('JJ'))

categories_gram = ['Noms', 'Verbes', 'Adjectifs']
comptes_gram    = [nb_noms, nb_verbes, nb_adjectifs]
couleurs_gram = ['#3498db', '#f1c40f', '#e74c3c']

print(f'\nNoms : {nb_noms:,}  |  Verbes : {nb_verbes:,}  |  Adjectifs : {nb_adje

# Graphique à barres
fig, ax = plt.subplots(figsize=(7, 4))
barres_gram = ax.bar(categories_gram, comptes_gram,
                     color=couleurs_gram, edgecolor='black', linewidth=0.8, widt

for barre, val in zip(barres_gram, comptes_gram):
    ax.text(barre.get_x() + barre.get_width() / 2, barre.get_height() + max(comp
            f'{val:,}', ha='center', va='bottom', fontsize=12, fontweight='bold'

ax.set_title('Fréquence des catégories grammaticales', fontsize=14, fontweight='
ax.set_ylabel('Nombre de tokens', fontsize=12)
ax.set_ylim(0, max(comptes_gram) * 1.15)
ax.spines[['top', 'right']].set_visible(False)
plt.tight_layout()
plt.show()
```

```
Étiquetage POS en cours...

Noms : 138,344  |  Verbes : 71,934  |  Adjectifs : 71,470
```


Fréquence des catégories grammaticales
Nombre de tokens
0
20000
40000
60000
80000
100000
120000
140000
138,344
71,934
71,470
Noms
Verbes
Adjectifs

In [ ]: